

Final Product and Order Domain Model

Approved refinement before implementation

Outcome

The approved service mapping remains Product Service, Order Service, and Order Activity Service. The domain model is now simplified: ClaimType has no replacement, Order contains no type field, ProductStatus represents availability directly, and Order Service accepts an order only when Product Service returns an existing product with AVAILABLE status.

No source code, configuration, database migration, or infrastructure file has been changed. This document is the final model proposal for approval.

Confirmed decisions

OrderType	Do not create it. Remove ClaimType without replacement.
ProductCategory	Keep it as the direct domain replacement for CoverageType.
ProductStatus	Use AVAILABLE, OUT_OF_STOCK, and DISCONTINUED.
Order eligibility	The referenced product must exist and its status must be AVAILABLE.
Architecture	Keep the existing Spring Boot, Eureka, Config Server, Gateway, OpenFeign, PostgreSQL, Flyway, Kafka, and DynamoDB topology.
Scope boundary	Do not add inventory quantities or any new service, component, or architectural pattern.

1. Final Product model

Product replaces Policy as the independently persisted master-data entity. Its status is an availability decision used directly by Order Service. The field set stays close to the current Policy shape to minimize implementation risk.

Product entity fields

Field	Java type	Persistence / role
id	UUID	Primary key generated with GenerationType.UUID.
sku	String	Required and unique product identifier; replaces policyNumber.
sellerId	UUID	Required owner or seller reference; replaces customerId in Policy.
name	String	Required product name; replaces vehicleVin.
category	ProductCategory	Required enum stored as a string; replaces CoverageType.
price	BigDecimal	Required positive product price; replaces coverageLimit.
status	ProductStatus	Required availability enum stored as a string.
availableFrom	LocalDate	Required start date; replaces effectiveDate.
availableUntil	LocalDate	Required end date; replaces expirationDate.
createdAt	LocalDateTime	Creation timestamp managed by Hibernate.

Final Product enums

```
public enum ProductStatus {  
    AVAILABLE,  
    OUT_OF_STOCK,  
    DISCONTINUED  
}  
  
public enum ProductCategory {  
    ELECTRONICS,  
    CLOTHING,  
    HOME_AND_GARDEN  
}
```

ProductCategory is intentionally a small starter taxonomy. Its values are a business taxonomy choice and can be revised without changing the service architecture.

2. Final Order model

Order replaces Claim as the transactional entity. It references one product and one customer, keeps the monetary total and lifecycle status, and deliberately omits ClaimType and any OrderType replacement.

Order entity fields

Field	Java type	Persistence / role
id	UUID	Primary key generated with GenerationType.UUID.
productId	UUID	Required Product Service reference; replaces policyId.
customerId	UUID	Required customer reference, preserved from Claim.
orderNotes	String	Required text with the existing 500-character limit; replaces description.
totalAmount	BigDecimal	Required positive monetary value; replaces amountRequested.
status	OrderStatus	Required order lifecycle enum stored as a string.
createdAt	LocalDateTime	Creation timestamp managed by Hibernate.

Final Order enum

```
public enum OrderStatus {  
    PLACED,  
    PROCESSING,  
    COMPLETED,  
    CANCELLED  
}
```

There is no OrderType enum and no orderType column, request field, response field, event field, mapper assignment, or DynamoDB attribute.

My recommendation on totalAmount

Because this preserved model references one product and introduces no quantity, Order Service should set totalAmount from the ProductResponse price returned by Product Service. The client should not be trusted to choose the stored order total. This requires no new component or pattern and keeps pricing consistent with the authoritative Product Service.

3. Order eligibility and failure behavior

Order creation keeps the existing synchronous validation structure. The current PolicyClient becomes ProductClient, and the existing exception-handling pattern is renamed rather than redesigned.

Required creation sequence

- Receive productId, customerId, and orderNotes in CreateOrderRequest.
- Call Product Service through OpenFeign and Eureka using productId.
- If Product Service returns 404, throw ProductNotFoundException and return HTTP 404.
- If Product Service cannot be reached, throw ProductServiceUnavailableException and return HTTP 503.
- If product.status is OUT_OF_STOCK or DISCONTINUED, throw ProductUnavailableException and return HTTP 409.
- Continue only when product.status is AVAILABLE.
- Create the Order with totalAmount set from product.price and status set to PLACED.
- Persist the Order to PostgreSQL and publish ORDER_CREATED to Kafka.

Eligibility rule

```
ProductResponse product = productClient.getProductById(request.productId());

if (product.status() != ProductStatus.AVAILABLE) {
    throw new ProductUnavailableException(request.productId());
}

Order order = OrderMapper.toEntity(request);
order.setTotalAmount(product.price());
order.setStatus(OrderStatus.PLACED);
```

This is an availability check only. It does not reserve stock, decrement inventory, or guarantee availability after the check.

4. Event and activity model

Removing ClaimType also removes it from the asynchronous contract. The producer-side and consumer-side event records must remain structurally identical.

Final OrderCreatedEvent fields

Field	Java type	Persistence / role
eventId	UUID	Unique event identifier.
eventType	String	ORDER_CREATED.
orderId	UUID	Identifier of the persisted order.
productId	UUID	Referenced product identifier.
customerId	UUID	Customer placing the order.
totalAmount	BigDecimal	Stored order total derived from product price.
occurredAt	LocalDateTime	Event creation timestamp.

DynamoDB activity attributes

- orderId as the partition-oriented domain identifier
- eventKey composed from occurredAt and eventId, preserving the current strategy
- eventId and eventType
- productId and customerId
- totalAmount
- occurredAt
- No orderType attribute

Kafka contract

- Topic: order-events
- Message key: orderId as a string
- Event type: OrderCreatedEvent
- Consumer: order-activity-service
- Serialization approach: preserve the existing Spring Kafka JSON configuration

5. Final scope and design thoughts

Why this model fits the existing project

- It preserves the same two-service synchronous dependency: Order Service validates Product Service through OpenFeign.
- It preserves database-per-service ownership with PostgreSQL and Flyway.
- It preserves the Kafka-to-DynamoDB activity pipeline.
- Removing ClaimType reduces accidental insurance terminology instead of forcing an artificial e-commerce concept.
- ProductStatus now expresses exactly what Order Service needs to decide: whether the product can be ordered.
- Using product.price as totalAmount avoids trusting a client-provided total without adding another service.

Explicitly excluded

- Inventory quantities or stock reservation
- inventory-service
- Payments or payment-service
- Carts
- Shipping or fulfillment
- Authentication or authorization
- Sagas, CQRS, or event sourcing
- Redis or any other new datastore
- Any other new architectural component or integration pattern

Preserved technology stack

Spring Boot	Eureka	PostgreSQL
Spring MVC	Config Server	Flyway
OpenFeign	API Gateway	Kafka
Jakarta Validation	Spring Cloud LoadBalancer	DynamoDB

Implementation remains paused. This document records the final proposed entity and enum model after the requested adjustments.